AGENDA 4

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE II

Expediente

Autores:

TIAGO ANTONIO DA SILVA

Revisão Técnica:

KELLY CRISTIANE DE OLIVEIRA DAL POZZO

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

Imagine que você está desenvolvendo um aplicativo de compras e deseja permitir que o usuário encontre rapidamente um produto na lista digitando seu nome. Como podemos implementar essa funcionalidade de busca de forma eficiente?"

Aprofundando os conhecimentos no .NET MAUI e explorando mais recursos do SQLite e do código-fonte previamente implementado, nesta agenda vamos recuperar os dados inseridos no SQLite, e para isso vamos explorar recursos da interface gráfica XAML e métodos especiais da plataforma .NET.

Também vamos implementar a funcionalidade de busca instantânea, isso significa que conforme o usuário for digitando será possível realizar a busca diretamente no SQLite. Para isso, vamos conhecer o elemento SearchBar e o evento TextChanged.

Elemento XAML: SearchBar

O SearchBar no XAML é um controle de interface do usuário (UI) que permite inserir texto para realizar uma pesquisa dentro de um aplicativo. Esse controle é utilizado para criar campos de busca eficientes e interativos, onde os usuários podem procurar por informações em listas, bancos de dados, no nosso caso, no SQLite. O SearchBar conta com várias propriedades e eventos que permitem personalizar seu comportamento. Ele pode ser usado em diversas plataformas, incluindo Android, iOS, macOS e Windows, e oferece uma interface simples e intuitiva para que os usuários encontrem rapidamente o que procuram.

Propriedades do SearchBar

No SearchBar temos duas principais propriedades:

- **Placeholder** (<SearchBar Placeholder="Digite para buscar" />): utilizada para definir o texto de espaço reservado (exibido quando o campo está vazio) que ajuda a informar o usuário sobre o que ele pode buscar, melhorando a experiência do usuário.
- **TextChanged** (<SearchBar TextChanged="OnSearchTextChanged" />): Evento disparado sempre que o texto da SearchBar for alterado, sendo útil para realizar uma pesquisa dinâmica conforme o usuário digita a informação.

Evento do SearchBar

Entre os eventos do SearchBar temos os principais:

- **SearchButtonPressed** (<SearchBar SearchButtonPressed="OnSearchButtonPressed" />): o evento é disparado quando o usuário pressiona o botão de pesquisa (geralmente a tecla Enter ou o ícone de

pesquisa), podendo ser usado para iniciar uma pesquisa completa.

- **TextChanged** (<SearchBar TextChanged="OnSearchTextChanged" />): Acionado quando o texto na SearchBar muda, sendo utilizado para realizar uma busca dinâmica ou atualizar a interface do usuário conforme o texto for alterado. Esse evento é útil quando você deseja reagir à digitação do usuário e realizar uma ação, como atualizar os resultados da pesquisa dinamicamente à medida que o usuário digita a informação.

Argumentos do Evento

O evento TextChanged usa o tipo TextChangedEventArgs como argumento. Essa classe contém duas propriedades principais:

- **OldTextValue:** O texto anterior, antes da alteração feita pelo usuário.
- **NewTextValue:** O novo texto após a alteração feita pelo usuário.

Um exemplo de utilização desses valores é comparar a mudança do texto ou executar uma ação específica, como filtrar dados ou realizar uma pesquisa.

Como Funciona?

Quando o usuário digita ou apaga algo na SearchBar, o evento TextChanged é disparado. Em seguida, o manipulador do evento (OnSearchTextChanged no exemplo acima) recebe dois parâmetros, sendo:

- OldTextValue: O texto antes da mudança.
- NewTextValue: O texto após a mudança.

Os valores podem ser utilizados para comparar e realizar ações em tempo real, como atualizar uma lista de resultados ou filtrar dados enquanto o usuário digita a informação.

Usos Comuns do Evento TextChanged

Pesquisa Dinâmica (Live Search): Uma das implementações mais comuns é a pesquisa dinâmica, onde os resultados da busca são filtrados enquanto o usuário insere a informação. Isso pode ser feito usando o evento TextChanged para atualizar a lista de resultados conforme o texto é alterado.

Validação de Texto: Outra aplicação possível é usar o evento TextChanged para realizar validações em tempo real no texto inserido. Por exemplo, você pode verificar se o texto inserido atende a determinados critérios (como comprimento mínimo ou formato).

ListView

A ListView é um dos controles mais utilizados no .NET MAUI para exibir listas de dados de maneira eficiente e interativa. Ele permite que você mostre uma coleção de itens em uma interface de usuário, proporcionando várias funcionalidades de personalização e interação, como navegação, seleções e ações em itens.

Sendo altamente configurável, a ListView oferece suporte para exibir dados simples, como texto, ou até mesmo elementos mais complexos, como imagens, botões ou controles personalizados. Vejamos as principais propriedades da ListView:

- **ItemsSource:** utilizada para vincular a ListView a uma coleção de dados. Pode ser uma lista de objetos, como uma lista de strings, uma lista de objetos complexos ou até mesmo dados vindos de um banco de dados (como o SQLite) ou API.
- **ItemTemplate:** define como cada item da lista será exibido. Pode ser um controle simples como TextCell, ou você pode usar um DataTemplate mais complexo, com vários controles dentro dele.
- **SelectedItem:** permite capturar o item selecionado na lista. Podendo ser utilizada para interagir com o item, como abrir uma página detalhada ou realizar outras ações.
- **IsPullToRefreshEnabled:** permite habilitar a funcionalidade de "arrastar para atualizar", que é um gesto comumente usado em aplicativos móveis para atualizar os dados da lista.

Eventos Comuns da ListView

A ListView conta com dois elementos comuns:

- **ItemSelected:** evento acionado quando um item da lista é selecionado. Ele é comumente usado para navegar para uma página de detalhes ou realizar outra ação no item selecionado.
- **Refreshing:** esse evento é disparado quando o gesto "arrastar para atualizar" é executado. Ele é frequentemente usado para recarregar os dados da lista.

ContextActions

As ContextActions da ListView são ações de contexto que podem ser ativadas por um gesto de deslizar (swipe) em um item da lista, proporcionando uma maneira intuitiva de oferecer opções de interação, como editar, excluir, ou realizar outras ações associadas ao item. Essas ações aparecem geralmente ao deslizar o item para a esquerda ou para a direita, dependendo da plataforma e da configuração.

Normalmente, as ContextActions são utilizadas para adicionar botões ou menus com opções rápidas, como Excluir, Editar, Favoritar, entre outras. Essas ações são definidas dentro de um DataTemplate, no qual podemos adicionar a propriedade ContextActions a um item individual para definir as opções.

ObservableCollection

A ObservableCollection<T> é uma classe que herda de Collection<T> e implementa a interface INotifyCollectionChanged, permitindo que a coleção notifique automaticamente a interface do usuário sobre alterações na sua estrutura, como adição, remoção ou modificação de itens. Isso é particularmente útil em aplicativos como os feitos com .NET MAUI, onde frequentemente precisa atualizar a interface do

usuário em resposta a mudanças nos dados.

A ObservableCollection<T> oferece três tipos de notificações principais por meio do evento CollectionChanged:

- **Adicionar um Item (Add):** quando adiciona um item à coleção, a interface do usuário será automaticamente atualizada para refletir a mudança.
- **Remover um Item (Remove):** quando remove um item, a interface do usuário também será atualizada.
- **Modificar um Item (Alterações nas propriedades de um item):** se os itens na coleção são objetos complexos (como Produto), e se altera uma propriedade de um item (como Preco ou Nome), a interface do usuário será atualizada automaticamente, desde que o objeto também implemente INotifyPropertyChanged. Caso contrário, a UI não será notificada sobre as mudanças nas propriedades de um item.

Por que usar ObservableCollection<T> em vez de List<T>?

Se utilizarmos uma List<T> para armazenar os produtos do aplicativo, ao adicionar ou remover um item, a interface do usuário não detectaria automaticamente essa mudança. Isso exigiria uma atualização manual da interface do usuário, tornando a aplicação menos eficiente. Já com a ObservableCollection<T>, qualquer alteração na lista é refletida imediatamente na tela, sem a necessidade de reatribuir o ItemsSource.

Método OnAppearing

O método OnAppearing() é um dos métodos de ciclo de vida de uma página que é chamado sempre que a página está prestes a aparecer na tela. Frequentemente é utilizado para realizar tarefas que precisam ser executadas cada vez que a página se torna visível para o usuário, como inicializar dados, atualizar a interface do usuário ou executar animações.

Sendo parte do ciclo de vida da página, o método OnAppearing() é chamado automaticamente pela Framework quando a página está prestes a ser exibida. Isso ocorre sempre que a página está sendo mostrada, como ao navegar para ela ou após ela ter sido ocultada temporariamente (por exemplo, em uma navegação de empilhamento de páginas).

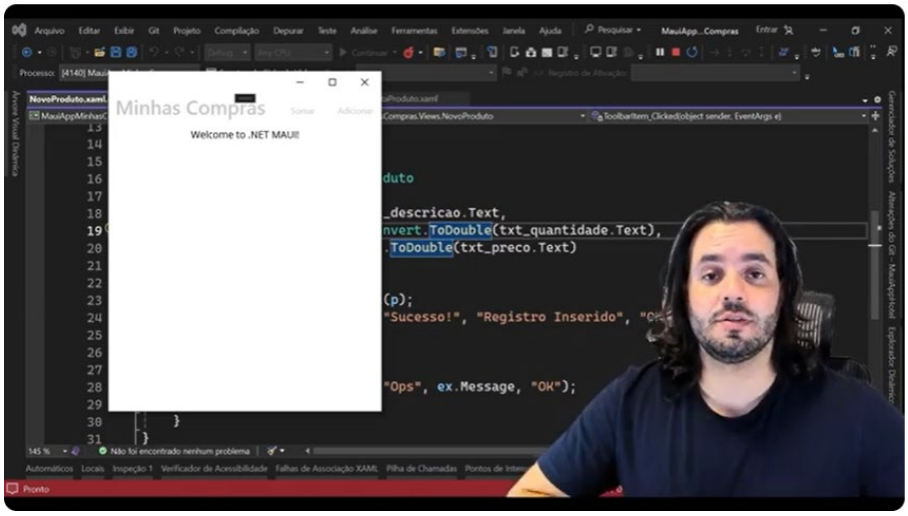
Quando o OnAppearing() é Chamado?

Após a navegação para a página: Quando navegamos para a página a partir de outra página, o OnAppearing() será chamado depois que a navegação for concluída e a página for apresentada na tela.

Após a página ser trazida de volta à frente: Se a página foi temporariamente oculta (por exemplo, se a navegação foi empilhada e você retornou à página anterior), o OnAppearing() será chamado novamente quando a página se tornar visível.

Após a rotação de tela ou alterações de layout: O método pode ser chamado novamente após uma rotação de tela ou outra mudança significativa no layout que força a página a ser redesenhada.

Assista o vídeo apresentando o passo a passo para criação do aplicativo:



Disponível em: https://www.youtube.com/watch?v=A27W2281aNo

CÓDIGO-FONTE DA AGENDA:

https://github.com/tiagotas/MauiAppMinhasCompras/tree/c19f4d3356b2ebd02e5ea392e5e7a162dcb0893f

Nesta agenda podemos compreender a importância da manipulação de interface para os aplicativos, garantindo que os dados sejam exibidos e atualizados de forma dinâmica e eficiente. A utilização do SearchBar para busca em tempo real, a ListView para exibição de listas e a ObservableCollection para atualização automática da interface do usuário são ferramentas essenciais para criar aplicativos interativos e responsivos. Com o conhecimento adquirido, agora você pode aprimorar ainda mais seus projetos, implementando buscas inteligentes, manipulando dados dinamicamente e garantindo uma experiência fluida para o usuário.